

From Brain to Browser

Turning the trained Leaper brain into a real web game you can play — how it works, what to expect, and the full roadmap, explained for a maker who is learning as they go.

You have spent months teaching a six-legged robot how to move. That work is done and it works. This guide is about the next chapter: giving that robot a game to live in — one where it chases, shoots, dodges, and can be shot back — all inside a web page, all in JavaScript.

Nothing here assumes you are a programmer. Every technical word is explained the first time it appears, and the short forms are always spelled out. Read it straight through and you should come away understanding the whole shape of the project: what we already have, what we are building, in what order, and how each piece fits with the others. Take your time — there is no rush, and every phase in the roadmap is something you will be able to see and play with your own hands.

PART ONE · WHERE WE ARE TODAY

What we already have

Everything inside your `E:\Leaper` folder is *already* a JavaScript web project. It uses a tool called **Three.js**, which draws three-dimensional graphics inside a normal web browser — that is what puts the robot, the ground, and the world on screen. So there is nothing to "move to another language." The game is already written in the language of the web. That is a real head start, and it means most of the hard graphics work is behind you.

Python — the other language you have been using — was never the game. It was the *workshop*. Python is where the brain was trained, using a method called **Reinforcement Learning (RL)**: a way of teaching by trial and error, where the robot tries something hundreds of thousands of times and slowly keeps whatever works. The end result of all that training is a

single small file — the brain — that takes in "what the robot senses" and gives back "what the robot should do."

There is one honest catch about today's setup, and it is the reason this whole project exists. Right now, the browser is not *running* the brain. It is playing back a **recording** — a large file (about 42 megabytes) that Python made earlier, like a video of the robot moving. A recording is perfect for studying training, but you cannot fight a recording. It does not know you are there and it never reacts. The heart of everything below is a single change: make the brain **think live, in the browser, many times a second**, so it responds to a real human player.

PART TWO · WHAT THE BRAIN CAN ACTUALLY DO

The brain's report card

Before building on top of the brain, it helps to know exactly how good it is and where its limits are — that way the game is designed around what it can genuinely do, not what we wish it could do. These are the real numbers from your champion run, **PPO_25**, measured over 100 fresh test attempts it had never seen.

89%

of targets reached successfully — nearly nine tries out of ten.

16

vision rays sweeping a wide 270° arc — its whole sense of the world.

~6%

of steps in collision — it mostly keeps itself clear of obstacles.

86%

of steps moving forward — it is decisive, rarely dithering or reversing.

In plain terms: **the brain is a confident, capable mover**. It heads toward its goal without hesitating, steers around most obstacles, and finishes the job the large majority of the time. That is more than enough to make a lively, believable opponent in a game.

Its honest limits are worth naming too. It "sees" only through those sixteen distance rays — it has no memory of a moment ago, and it was trained to *reach a target*, not to fight. That is completely fine, and in fact it shapes our whole plan: we let the trained brain do the one thing it is excellent at — **moving toward things and around obstacles** — and we add the fighting on top with simple hand-written rules, rather than asking the brain to learn something it was never taught. More on that shortly.

PART THREE · THE WORDS WE WILL USE

A plain-language glossary

Read this once and the rest of the guide reads smoothly. Each short form is spelled out here and then used freely.

The Brain · trained with PPO — Proximal Policy Optimization

The decision-maker you trained. **PPO** is just the name of the training *method* — the trial-and-error recipe that taught it. What you are left with is a small bundle of numbers that turns "what I see" into "what I do."

Its shape · MLP — Multi-Layer Perceptron

The *type* of network inside the brain. An **MLP** is the simplest, most standard kind — here just two small layers of 64. Being small and simple is a gift: it runs instantly, even in a browser. Your file is labelled `MlpPolicy` for exactly this reason.

No memory · LSTM — Long Short-Term Memory

A memory add-on that some brains use to remember earlier moments. An **LSTM** lets a network carry information forward in time. You tried it; it did not help, so the champion has none. Good news for us — a memoryless brain is far simpler to run live, because each frame stands entirely on its own.

What it senses · the Observation — 26 numbers

Every frame, the brain is handed 26 numbers: **16 vision rays** (each the distance to whatever it hits, spread across a 270° **Field Of View, or FOV**), plus a few facts about the target's direction and the robot's own speed and body. Those 26 numbers are the brain's entire world.

What it decides · the Action — 2 numbers

The brain answers with just two numbers: **throttle** (0 means stop, 1 means full speed ahead) and **turn** (−1 is hard left, +1 is hard right). That is the brain's entire "muscle output" — everything else follows from those two.

The travel format · ONNX — Open Neural Network Exchange

A universal save-file for brains that web browsers can read. We convert the Python brain into an **ONNX** file *once*, and it becomes portable — ready to travel out of Python and into the game.

The browser engine · [onnxruntime-web](#)

A small, free JavaScript library that actually *runs* the ONNX brain inside the web page — handing it the 26 numbers and reading back the 2, dozens of times a second.

The combat rules · Behavior Tree, and its cousin the FSM — Finite State Machine

Hand-written "if this, then that" logic that decides *when to shoot or run* — no training involved. A **Behavior Tree** is a checklist. An **FSM** is its simpler cousin, where the leaper is in exactly one mode at a time. Both are explained with pictures further on.

PART FOUR · GETTING THE BRAIN INTO THE BROWSER

The brain's journey

This is the single most important idea in the whole project, so it is worth slowing down for. There are two very different worlds. On the left is a one-time step you do in Python, on your own machine. On the right is the loop that runs forever inside the player's browser while the game is being played.

DONE ONCE · IN PYTHON

Trained brain
leaper_ppo.zip

export

Portable brain
leaper.onnx

ship in game

Loaded by
onnxruntime-web

LIVE · EVERY FRAME · IN THE BROWSER

Observation
26 numbers in

feed in

The brain thinks
the MLP runs

move legs → look again → repeat

Action
2 numbers out

read out

Read it left to right. The Python step happens once, on your machine. Everything on the right repeats dozens of times a second inside the player's browser: build the 26 numbers, run the brain, get 2 numbers back, move the legs, look again.

The beautiful part is how little needs to cross from Python into the game: just that one `leaper.onnx` file. Once it is exported, you can build and change the game for weeks without opening Python again. Python only comes back into the picture on the day you decide you want a *smarter* brain — then you retrain, re-export, and drop in a new file.

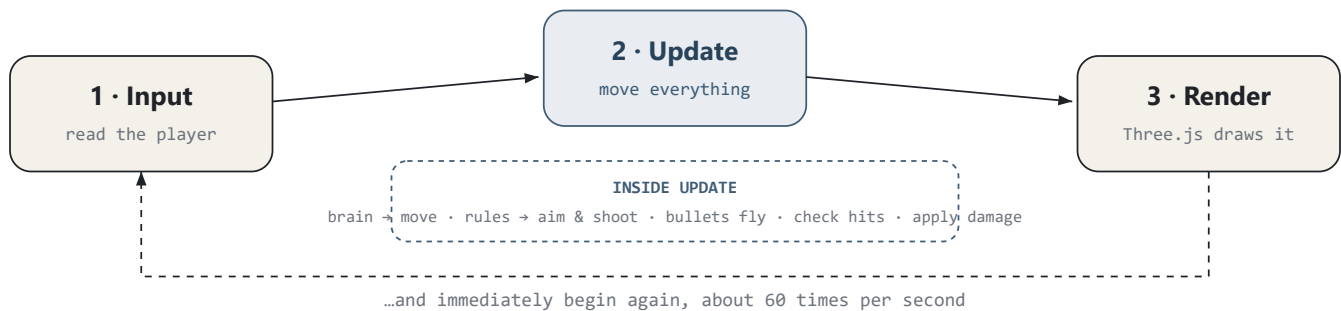
THE ONE THING TO GET EXACTLY RIGHT

The brain expects those 26 numbers to be built in JavaScript **exactly** the way Python built them — the same order, the same scaling. If they are even slightly different, the brain behaves as if it were dizzy. So the most important single task in this whole plan is copying Python's "recipe" for the 26 numbers over to JavaScript, faithfully. It is not difficult work; it simply has to be precise. This is the honest speed-bump of using the real brain — worth it, but worth knowing about up front.

PART FIVE · HOW A WEB GAME ACTUALLY RUNS

One loop, repeating forever

Underneath every game — from the simplest phone game to the biggest console title — is a single loop that repeats about sixty times every second. Each time around, it does three things: it **reads the player**, it **updates the world**, and it **draws the new picture**. Then it does it all again. When this happens sixty times a second, your eye sees smooth, living motion.



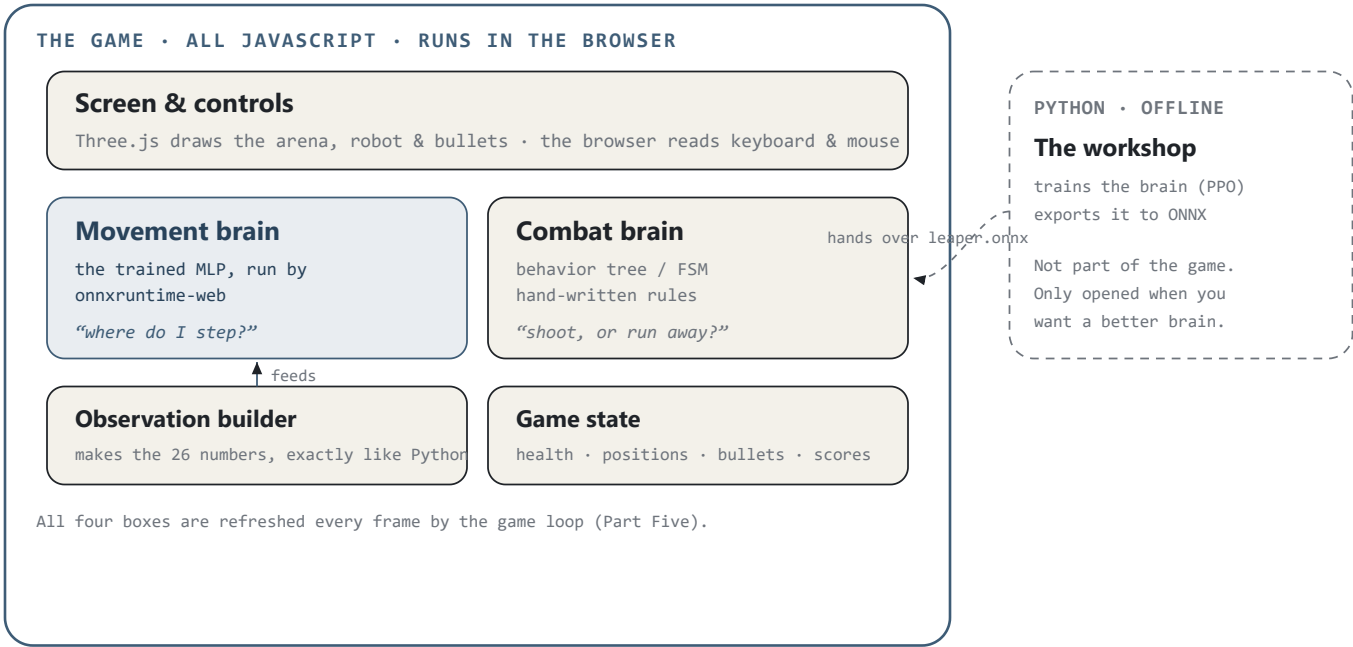
Where the thinking lives. Both of the leaper's "brains" — the trained **movement brain** and the hand-written **combat rules** — are simply steps inside the Update stage. Everything the player finally sees comes out of the Render stage.

This is genuinely all a game is, at its core. Once you can picture this loop, the rest of the project is just deciding what goes inside the "Update" step — and that is exactly where our two brains and all the fun live.

PART SIX · THE WHOLE TOOLBOX, IN ONE PICTURE

What does what

This is the map you asked for — every tool and the exact job it holds. The important thing to notice is that **the whole game is one thing: JavaScript, running in the browser.** Inside it there are two "brains" with clearly separate jobs. Python sits off to the side, not part of the game at all, quietly available for when you want to improve the brain.



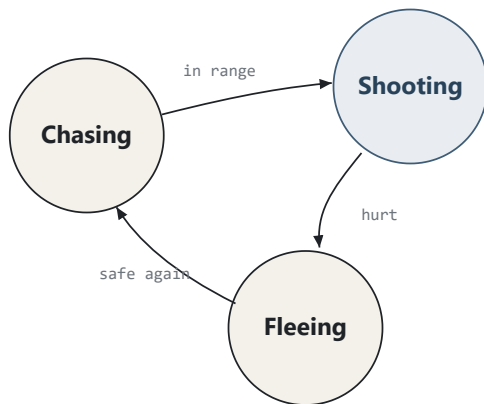
The rule to carry with you: the whole game is JavaScript, and Python only ever hands over a single file. Two brains, clearly divided — one moves the body, one decides the fighting — and everything is refreshed each frame by the loop from Part Five.

PART SEVEN · THE FIGHTING LOGIC

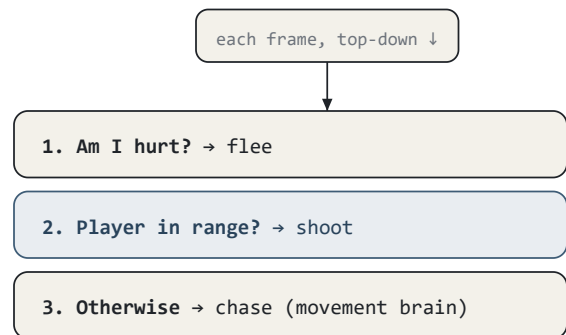
Behavior tree versus state machine

Here is the good news you already sensed: you do **not** need reinforcement learning to decide when the leaper shoots. Teaching a brain to fight would be an enormous, painful effort for a web game. Instead we simply *write the rules* by hand, in plain "if this, then that" form, and tune them by playing until they feel right. There are two common ways to write those rules, and it helps to see both.

STATE MACHINE – one mode at a time



BEHAVIOR TREE – a priority checklist



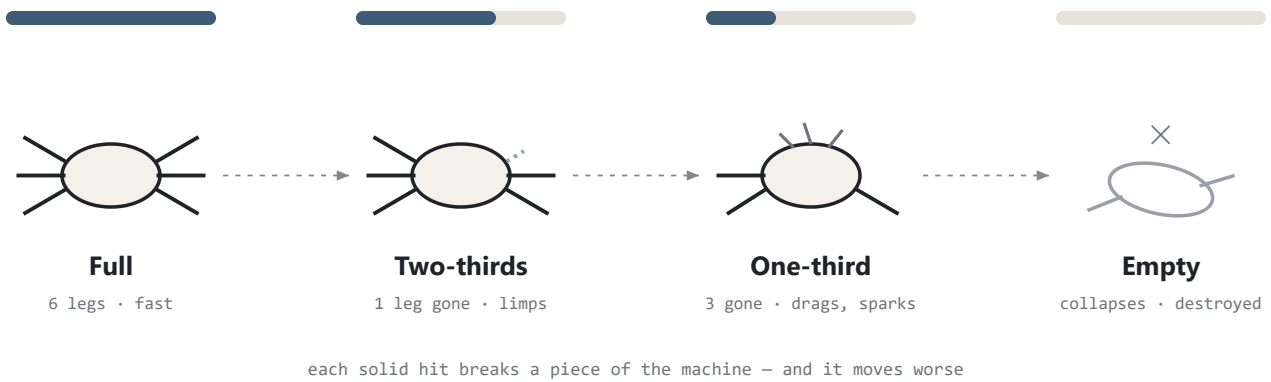
The difference in one line: a **state machine** is a map of modes with doors between them; a **behavior tree** is a priority list checked top to bottom every frame, where the highest rule that is true wins. For the leaper, **start with the behavior tree on the right** — it is the easiest to read, and the easiest to grow later.

Notice the lovely division of labour in that checklist: rules 1 and 2 handle the *fighting* decisions, and rule 3 simply hands control back to the trained movement brain to do the *chasing*. Your months of reinforcement-learning work power that last line, while the fighting on top is just a few readable rules you can tune in an afternoon.

PART EIGHT · MAKING DAMAGE SOMETHING YOU CAN SEE

Health you can see and feel

Your instinct here was exactly right, and it is a lovely one: a player should not need to read a number to feel that they are winning. And you have a gift most games do not — your robot is built from *six separate legs*. That means the robot's own body can be the health bar. We do not just drain a number; we damage the machine itself.



The body is the health bar. A small bar still shows the number, but the real feedback is physical: legs blow off, the walk becomes a limp, sparks and smoke appear. The player sees and feels the damage without reading a thing.

The trick to good "game feel" is to stack several of these cues at once, so a hit is impossible to miss:

- **A floating health bar** above the leaper that shrinks as it takes damage.
- **A hit flash** — the whole robot flashes bright for a split second on every hit. This alone gives the instant, satisfying feeling of "I connected."
- **Damage numbers** — a small "-12" popping off the body, if you want an arcade feel. Optional.
- **Legs detaching** — the strongest cue of all, and effectively free, because you already built the robot from six separate legs.
- **Movement getting worse** — with fewer legs the robot moves slower and limps. The brain still drives; it simply has weaker muscles now.
- **Sound and a small screen shake** on the final, killing blow.

A QUIET PIECE OF MAGIC

Because movement comes from the trained brain, and damage just *weakens the muscles* (a lower top speed, a slower turn), you get believable "wounded" behavior for free — the brain keeps trying its best plan with a broken body, exactly the way a real injured animal would. You never have to program "limping." It simply emerges.

Two players — the honest answer

Yes — but *where* the two players sit changes the difficulty enormously, and it is worth being honest about that from the start. There are two very different roads, and they are not the same size of task at all.

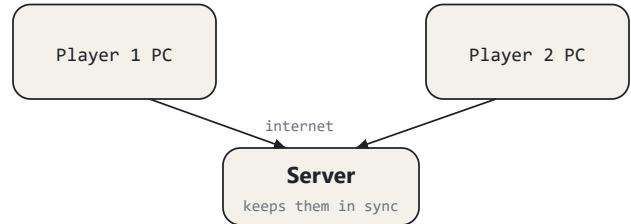
A • SAME SCREEN — easy, start here

One computer

P1: keyboard • P2: gamepad
one browser, one game loop

No internet, no server. Works today.

B • ONLINE — hard, much later



Same-screen is a weekend; online is a project of its own. Online play needs a server, a way to match players together, and constant syncing of two computers across the internet — with all the lag, dropped connections and cheating that brings.

My honest recommendation: ship **one human versus the leaper** first. It is the simplest possible version, and — most importantly — it is the thing that proves the trained brain really works in a live game. Add **same-screen two-player** when you want a friend to join on the couch. Treat **online play** as a distant, separate adventure for much later; it is a whole discipline in itself, not just another feature to add.

PART TEN • THE BUILD ORDER

The complete roadmap

Here is the whole journey, in the order I would build it. The single most important principle: **every phase is something you can see and play.** You never spend weeks in the dark. Each step ends with a visible, working thing — which is exactly what keeps a project alive when you are learning as you go.

1 Wake the brain up

Export `leaper_ppo.zip` into a `leaper.onnx` file in Python (one small script, run once), then load it in the browser with `onnxruntime-web`. The real work here is rebuilding Python's "26 numbers" recipe in JavaScript, exactly. When it is done, the leaper chases a target *live*, reacting in real time — no more replaying a recording. This is the moment your months of training first come alive on the web.

The one genuinely hard step. The "exact 26 numbers" gotcha lives here — get past this and the rest is downhill.

2 Give the player a body and a gun

Let yourself move with the keys, aim with the mouse, and click to fire bullets into the arena. No enemy behavior yet — just you, moving and shooting, learning how the controls feel. Small and satisfying, and it involves no brain at all.

Pure, ordinary JavaScript. A gentle step after the hard one.

3 Arm the leaper with the behavior tree

Add the chase → shoot → flee checklist from Part Seven. Now the leaper makes real combat decisions layered on top of its brain-driven movement. It will start to feel like an opponent with intentions, not just a wanderer.

Hand-written rules, tuned by playing until the leaper feels fair but dangerous.

4 Bullets, hits and health

Make bullets actually check whether they hit; make hits deal damage; make damage trigger the visible feedback from Part Eight — the flashes, the legs blowing off, the limp. This is the phase where someone can finally win or lose. It becomes a game.

The turning point, where all the earlier pieces click together into something you play to win.

5 Make it feel good

Add the finishing touches: health bars, a win-and-lose screen, a restart button, a score, sound, a little screen shake. Then play it over and over and tune the numbers — firing range, fire rate, damage, speed — until it is genuinely, honestly fun. This is usually the longest and the most enjoyable phase.

Polish and play-testing. The part where a working prototype becomes a real little game.

6 Later, only if you want it

Same-screen two-player from Part Nine. A smarter, retrained brain from the Python workshop. And, someday far off, online play. None of these are needed to have a finished, shareable game — they are optional adventures for when you are ready for them.

Optional. Your game is complete and worth sharing well before this point.

IN CLOSING

The shape of the whole thing is simpler than it first looks. You already have the hardest, most impressive part — a trained brain that moves well, and a 3D world to move it in. From here it is one careful step (waking the brain up in the browser), then a series of small, visible, satisfying additions on top. Take them one at a time, play with each before moving on, and you will end up with a real game built around a real brain you taught yourself. Whenever you are ready, the natural first move is Phase 1 — and we can take it together, slowly, one piece at a time.